

Authentication API Report

1.1 Overview

The authentication service employed in Databytes Intelligent IoT this semester uses:

- Express.js
- JSON-based temporary user storage
- JWT authentication
- Bcrypt password hashing
- Authentication middleware
- Role Based Access Control (RBAC)
- Layered architecture (routes → controller → service → repository)

1.2 System Architecture

Layer	Responsibility	Authentication Flow
Routes	Define available API endpoints /auth/login and /auth/register Maps controller function requests Applies middleware.	Client Request ↓ Routes Layer ↓ Controller Layer ↓ Service Layer ↓ Repository Layer ↓ Temporary JSON Storage
Controller	Handles HTTP req/res Returns JSON responses	
Service	Register user Authentication user Validate password Generate JWT token Role Based Access Control logic	
Repository	Data/ storage access Read, write and find users from JSON storage users.json	
Middleware	Protect routes Validate JWT token Check user roles Access permission	

2.1 Authentication Components

2.2 Routes Layer

Purpose

Defines API endpoints and maps requests to controllers.

Method	Endpoint	Description
POST	/auth/register	Register new user
POST	/auth/login	Login existing user
GET	/auth/protected	Access protected route
GET	/auth/hash-test	Convert into secure hash with bcrypt
GET	/auth/compare-test	Validates stored password
GET	/auth/test-token	Get JWT token

2.2 RBAC user groups

User Role	Description
user	Standard authenticated user
admin	Full administrative access
test	Test user

2.4 Controller Layer

Handles HTTP request and response logic.

Responsibilities

- Reads request body
- Calls authentication service
- Returns JSON responses
- Handles HTTP status codes

2.5. Service Layer

Contains core authentication business logic. The service layer is independent from Express routes.

Responsibilities

- Register users
- Hashes password
- Validate passwords
- Generate JWT tokens
- Handle failed login attempts
- Lock accounts after repeated failures

2.6 Repository Layer:

Handles data storage operations.

Current Storage - Temporary JSON storage: /data/users.json.

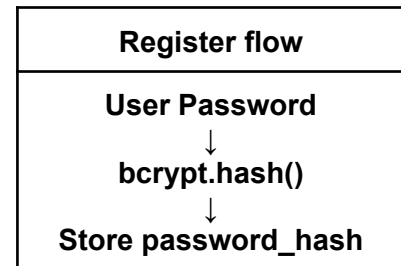
The repository layer allows migration to SQL databases later without changing the database files such as the controllers, routes and other services. Only the repository implementation would need to change.

3. Register and Login Flow

3.1 register flow

The authentication service:

1. Checks if the email already exists
2. Hashes the password using bcrypt
3. Creates a new user record
4. Stores the user in temporary JSON storage
5. Returns user information
6. User registers with username and password



1. **Endpoint:** POST /auth/register
2. **Endpoint:** GET /auth/hash-

Registers a new user. The user's password is hashed into secure storage. Using bcrypt, the plaintext is securely hashed and stored into the database

Example:

```
{
  "email": "test@example.com",
  "password": "Password1"
}
```

Hashed output:

```
{
  "id": "user-001",
  "username": "admin",
  "password_hash": "$2b$10$abc123...",
  "role": "admin"
}
```

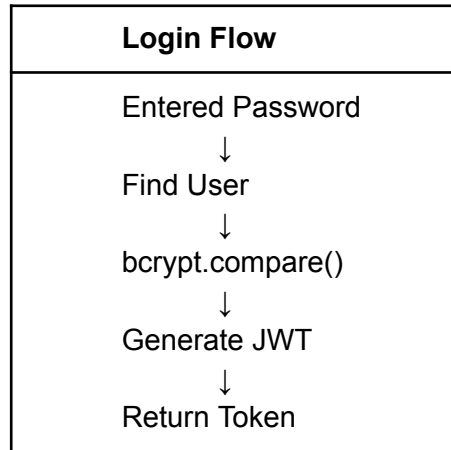
Successful Registration Response:

```
{
  "user_id": "9e7a9d55-1f34-4d91-b7d3-fd2bb84b8b2c",
  "email": "test@example.com"
}
```

If a user exists: If the associated email exists in the database, registration is declined and it throws an error.

3.2 Login Flow and Token Generation

Here the user's password during login is compared to the hash password previously stored according to user credentials. It will reject login if the credentials do not match. If validated, it generates a JWT token.



3. **Endpoint:** /auth/login

4. **Endpoint:** /auth/compare-test

Login Request Body:

```
{
  "email": "admin",
  "password": "Password1"
}
```

5. **Endpoint:** /auth/test-token

If the user password is successful then the access token is generated for the client.

Example token:

```
{
  "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
}
```

Example token payload:

```
{
  "user_id": "9e7a9d55-1f34-4d91-b7d3-fd2bb84b8b2c",
  "email": "test@example.com",
  "role": "user"
}
```

6. Successful Login Response

Response

```
{
  "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "user": {
    "user_id": "9e7a9d55-1f34-4d91-b7d3-fd2bb84b8b2c",
    "email": "test@example.com"
  }
}
```

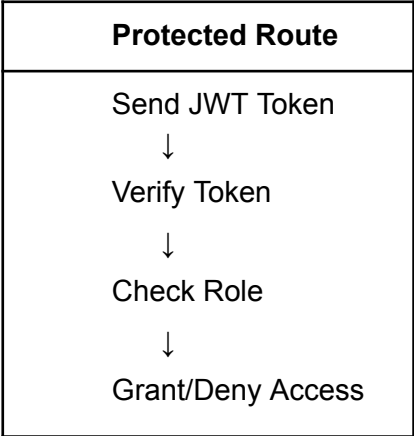
3.3 Protected Route Authentication

The route forwards the request to `authMiddleware` to verify the token and obtain the user identity role. According to the user role, it will either provide limited or admin access determined by user access permissions.

Endpoint: `/auth/protected`

Examples:

```
{
  "message": "Authenticated",
  "user": {
    "user_id": "9e7a9d55-1f34-4d91-b7d3-fd2bb84b8b2c",
    "email": "test@example.com",
    "role": "user",
    "iat": 1746752000
  }
}
```



Test example:

Token

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6Im
```

The authorization header will be automatically generated when you send the request. Re

Body 9 Headers 8 Raw 11

Body

▼ object {2}

message : Access granted

▼ user {4}

id : 1

role : admin

iat : 1778516370

exp : 1778519970

4. Protected Routes

Protected Routes:

A protected route requires a valid JWT token before access is granted.

Protected routes use: `authMiddleware`

4.1 Accessing Protected Routes

1. Send login request:

`POST /auth/login`

Copy the returned access token.

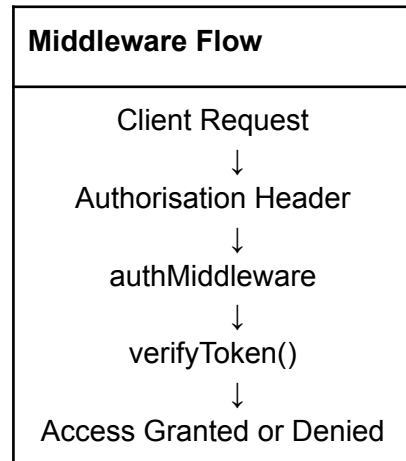
2. Add Authorisation Header

Example:

Authorisation: Bearer EXAMPLE_TOKEN

3. Send Protected Request

`GET /auth/profile`



4.2 Example Protected Request

Request

`GET /auth/user`

Authorisation: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...

Successful Protected Route Response:

```
{
  "message": "Authenticated",
  "user": {
    "user_id": "9e7a9d55-1f34-4d91-b7d3-fd2bb84b8b2c",
    "email": "test@example.com",
    "iat": 1746752000
  }
}
```

Access Protected Route Curl example:

```
curl http://localhost:3000/auth/profile \
-H "Authorization: Bearer EXAMPLE_TOKEN"
```

5. Error response format

Code	Description
INVALID_CREDENTIALS	Email or password is incorrect
USER_ALREADY_EXISTS	Email already registered
ACCOUNT_LOCKED	User account locked
NO_TOKEN	Missing authentication token
INVALID_TOKEN	JWT invalid or expired
INSUFFICIENT_PERMISSIONS	User lacks required role
LOGIN_FAILED	Generic login failure
REGISTER_FAILED	Generic registration failure

Frontend applications should:

- Use `success` to determine request outcome
- Display `error.message` to users
- Use `error.code` for logic handling and UI state

6. Security Features Implemented

- JWT authentication
- Password hashing with bcrypt
- Token verification middleware
- Failed login tracking
- Account locking
- Multi-user support
- Separation of concerns architecture

7. Future Improvements

- SQL database integration
- Refresh tokens
- Token revocation
- MFA / TOTP authentication
- API token persistence

8. Conclusion

The authentication system has been refactored into a modular layered architecture that:

- Removes authentication logic from the main server file

- Separates routes, controllers, services, and repositories
- Supports multiple users using temporary JSON storage
- Uses reusable JWT utilities
- Supports protected routes with middleware
- Can later migrate to SQL databases with minimal code changes